A single regular expression:

```
var re = /^http:\/\/localhost(:\d+)?$/i;
```

Matches all these whitelist values:

```
var originWhitelist = [
  "http://localhost",
  "http://localhost:1",
  "http://localhost:10",
  "http://localhost:100",
  "http://localhost:200",
  "http://localhost:300",
  "http://localhost:400",
  "http://localhost:1000",
  "http://localhost:1111",
  "http://localhost:2000",
  // And on and on and on...
;]
```

Figure 6.4 Regular expression versus a whitelist

number) can access the API. If you had hundreds of clients, each living on a different port, you can see how a single regular expression would be easier to manage than a whitelist with hundreds of entries, as shown in figure 6.4.

A regular expression is just one example of using a custom function to validate the origin. Table 6.2 shows more ways to validate the origin, along with the pros and cons of each.

Table 6.2 Pros and cons of validation techniques

Validation technique	Description	Pros	Cons
Whitelist	Maintains a list of valid origins	Clearly indicates which origins are allowed Works well for small lists of origins	Difficult to maintain as the list grows larger
Regular expression	Writes a regular expression that matches all the valid origins	Works well for a range of origins that follow a pattern	Need to verify that the regular expression doesn't accidentally match an invalid origin
Database query	Stores the list of valid origins in a database	Provides a central location for storing origin information Works well for a large number of origins without any common pattern Good for maintaining a consistent CORS policy across different servers	Database calls can be slow (may need a caching layer to speed up things)
Blacklist	The opposite of a whitelist: maintains a list of origins that aren't valid	Easier to maintain if you'd like to allow all but a few origins	Easy for clients to bypass (just pick a new origin that isn't in the blacklist)